

מסמך תיעוד וניתוח סיבוכיות

מגישות: ליאור פרניק ואלין לושבסקי

– AVLTree המחלקה

המחלקה מתארת עץ מאוזן, המחלקה מממשת פעולות הכנסה, חיפוש, מחיקה, ייצוא העץ למערך, חישוב גודל העץ ומציאת דרגה של צומת נתונה.

שדה המחלקה היחיד הוא root, והוא מייצג את השורש של העץ

הפונקציות המרכזיות הנמצאות במחלקה הן:

- Search
- Insert
- Delete
- Avl_to_array
- Size
- Rank
- Select
- Max_range
- Get_root

פונקציות העזר לפונקציות המרכזיות –

- Update_sub_tree
- Delete_node
- Fix_tree
- Rotation
- to_array

תיאור הפונקציות וניתוח סיבוכיות זמן הריצה שלהם:

1. הפונקציה Search –

הפונקציה מקבלת מפתח k ומחזירה את הצומת בעץ שמכילה מפתח זה, אם קיימת, אחרת מחזירה None, הפונקציה מבצעת חיפוש בינארי על העץ לחיפוש המפתח הנתון $\leftarrow O(\log n)$

2. הפונקציה **Insert** –
 הפונקציה מקבלת כקלט מפתח וערך. מייצרת צומת חדשה עם 2 בנים - צמתי דמה ומכניסה אותה תחילה כעלה (בעזרת חיפוש בינארי על מקום הכנסה) $\leftarrow O(\log n)$
 לאחר הכנסת הצומת, נקראת הפונקציה `fix_tree` שמטרתה לעדכן ולאזן את העץ בעזרת קריאות לפונקציה `rotation` במידת הצורך. תיעוד לפונקציות **fix_Tree, rotation** נמצא בהמשך (סעיף 9).
 ראינו בהרצאה שבפעולת הכנסה נדרשת פעולת איזון אחת לכל היותר $\leftarrow O(\log n)$
 סה"כ ביצענו שתי פעולות ב $O(\log n)$ ולכן הסיבוכיות הכוללת היא $O(\log n)$.
3. הפונקציה **delete** –
 מקבלת צומת למחיקה ובודקת כמה ילדים אם בכלל יש לו. בהתאם לכמות הילדים הצומת נמחק ע"י עיקוף הצומת או החלפתו עם העוקב שלו בעץ. לשם כך הפונקציה מופרדת ל2 מקרים ובהתאם קוראת לפונקציה `Delete_node` עם האב של הצומת שעתיד להמחק והצומת.
 במקרה ולצומת יש 2 ילדים, נחליף אותו בעוקב שלו ונעדכן את תת העץ בעזרת קריאה לפונקציה `update_sub_tree`, הלוקחת $O(1)$. תיעוד לפונקציה נמצא בהמשך (סעיף 8)
 3.1. הפונקציה **Delete_node** –
 הפונקציה מקבלת הורה וצומת, מבצעת את מחיקת הצומת שהתקבלה ועדכון שדות ההורה מצביע לבן חדש בהתאם למקרה, גודל תת העץ וגובה. $\leftarrow O(1)$
`Delete_node` קוראת ל `Fix_tree`.
 לכל היותר מתבצעות $\log n$ קריאות לפונקציה `rotation` (במסלול מהשורש עד להורה של הצומת שנמחק). כל רוטציה היא $O(1)$ ולכן סה"כ סיבוכיות $\leftarrow O(\log n)$.
 תיעוד לפונקציות **fix_Tree, rotation** נמצא בהמשך (סעיף 9).
 סה"כ סיבוכיות של `delete` $\leftarrow O(\log n)$.
4. הפונקציה **avl_to_array** –
 הפונקציה לא מקבלת קלט, מאתחלת מערך ריק וקוראת לפונקציה `to_array` ומחזיר את המערך שנוצר.
 4.1. הפונקציה **to_array** –
 הפונקציה מקבלת כקלט את שורש העץ ומערך ריק, ומבצעת סריקת in-order על כל המערך, בכל קריאה לפונקציה - מכניסים למערך `tuple` של (מפתח, ערך) עבור כל הצמתים, ולבסוף מוחזר מערך ממויין המייצג את העץ $\leftarrow O(n)$.
5. הפונקציה **size** –
 הפונקציה לא מקבלת קלט, מחזירה את גודל העץ בעזרת גישה לשדה גודל של שורש העץ $\leftarrow O(1)$
6. הפונקציה **rank** –
 הפונקציה מקבלת מצביע לצומת בעץ ומחזירה את הדרגה שלו. הפונקציה מחשבת את הדרגה ההתחלתית של הצומת לפי הדרגה של בנו השמאלי, ועולה במעלה העץ תוך כדי עדכון הדרגה. סה"כ בכל רמה ניגשים בודקים צומת אחת $\leftarrow O(\log n)$.

7. הפונקציה **select** –

הפונקציה מקבלת כקלט מספר i המייצג דרגה בעץ ומחזירה את הצומת בעץ שזוהי הדרגה שלה. הפונקציה קוראת ל-`select_rec` עם השורש i , ומחזירה את התוצאה.

7.1. הפונקציה **select_rec** –

הפונקציה היא פונקציה רקורסיבית המקבלת כקלט צומת ודרגה, בכל קריאה לפונקציה, מחושב הדרגה שלה (לפי שדה הגודל של הצומת), אם הצומת הנוכחית היא לא הרצויה, מתבצעת בדיקה הקובעת להמשיך לתת העץ הימני או השמאלי של הצומת הנוכחית. סה"כ מתבצעות $\log n$ קריאות רקורסיביות ובכל קריאה לוקחת $O(1)$ פעולות $\leftarrow O(\log n)$ סה"כ $\leftarrow O(\log n)$.

8. הפונקציה **Update_sub_tree** –

מקבלת צומת שתשמש כאב וצומת שתהיה בן. הפונקציה מעדכן את שדה האב של הצומת וצומת האב שקיבלנו ומעדכנת את הילד הימני או השמאלי תלוי במפתח להיות הצומת החדשה. כמו כן הפונקציה מעדכנת את שדות הגובה והגודל של האב והבן. $\leftarrow O(1)$.

9. הפונקציה **Fix_tree** –

מקבלת צומת אב של הצומת שהוכנס/נמחק ובודקת האם שונה ה-`BL` שלו: אם לא, עולים מעלה בעץ במסלול עד השורש ובודקים האם ישתנה ה-`BL` ומעדכנים את גודל וגובה הצומת.

אם כן, נקרא לפונקציה `rotations` ונחזיר את מספר הרוטציות שנעשו בה.

9.1. הפונקציה **rotations** –

הפונקציה מקבלת צומת אב של הצומת שמוחקים/מכניסים ומשתנה אופציונלי `rotate` שקובע איזה רוטציה תעשה. אם המשתנה ריק אז הפונקציה תחשב את סוג הרוטציה מתוך ה-`BL` של האב: אם `BL == 2` נבדוק איזה רוטציה לבצע לפי בן שמאלי אחרת לפי בן ימני. פונקציה כוללת `match-case` עבור סוג הרוטציה שהוא בעצם טפאל שמכיל במקום 0 את `BL` האב ומקום 1 את `BL` של הבן המתאים. בהתאם מתבצעת רוטציה שמאלה או ימינה או שניהם.

עד כה כל הבדיקות נעשות ב- $O(1)$

9.1.1. הפונקציות **rotate_left** ו-**rotate_right** –

הפונקציה מקבלת צומת שהוא שורש של תת עץ ומשנה את המצביעים שלו ע"י סיבוב ימינה או שמאלה בהתאמה, כפי שלמדנו בהרצאה וקוראת לפונקציה `Update_sub_tree` כדי לעדכן את תת העץ. $\leftarrow O(1)$

לאחר קריאת לרוטציה המתאימה נעדכן את שדות גודל וגובה הצומת שקיבלנו בהתחלה ונחזיר את כמות הרוטציות. נשים לב שכל הרוטציות מבצעות החלפת פוינטרים ולכן פועלות ב- $O(1)$. סה"כ סיבוכיות הזמן של הפונקציה $\leftarrow O(1)$

10. הפונקציה **get_root** –

הפונקציה מחזירה מצביע לשורש העץ $\leftarrow O(1)$

חלק תאורטי

1. מימוש ספירת המספר החילופים: בסוף כל חיפוש של מקום ההכנסה של הצומת החדש שמנו לב כי עבור הכנסה משמאל צריך לעשות הזזה של כל האיברים שהאנק שלהם גדול מהאנק של הצומת שהוכנס. לכן בשלב הכנסה מצד שמאל הוספנו לסכום ההחלפות הכולל את גודל העץ פחות האנק של הצומת החדש.
- מימוש חישוב ה-rank = גודל תת עץ ימני – גודל העץ. גודל תת עץ ימני זה תמיד גודל תת עץ שהמפתח של השורש שלו < ממפתח שרוצים להכניס (עלייה של $\log k$).

מספר סידור i	מספר חילופים במערך ממין-הפוך	עלות מיון AVL עבור מערך ממין-הפוך	מספר חילופים במערך מסודר אקראי	עלות מיון AVL עבור מערך מסודר אקראי
1	2467531	29205	617546	27663
2	9867903	86341	2428556	59668
3	39484941	220367	9720377	127838
4	157966425	511493	39080134	273525
5	631919025	1124573	157867862	585252

2. כמות ההחלפות במערך ממין בסדר הפוך: עבור על האיבר הראשון שמכניסים צריך לעשות $(1-n)$ החלפות. ועבור האיבר השני $(2-n)$ וכך הלאה. נשים לב כי קיבלנו סכום של סדרה חשבונית בעלת n איברים. לפי נוסחה: $\frac{(n-1) \cdot n}{2}$ החלפות.

כמות החיפושים: $\theta(n \log n)$

מצד אחד נחסום את עלות החיפושים מלמטה עי חישוב עלות החיפוש כאשר העץ חצי מלא. נשים לב בהכנסת חצי מהאיברים שנשארו כיוון שהמפתח שלהם קטן מכל הצמתים הנמצאים בעץ הנוכחי, נעלה מצומת המקסימום את כל גובה העץ שהוא $O(\log n/2)$. אז עבור הכנסת $n/2$ האיברים הנוותרים:

$$\frac{n}{2} \log\left(\frac{n}{2}\right) = \frac{n}{2} (\log n - 1) = \Omega(n \log n)$$

מצד שני נחסום את עלות החיפושים מלמעלה עי הגובה המקסימלי של עץ AVL עם n איברים. (נשים לב שבכל הכנסה נעלה עד השורש ונרד עד לעלה ולכן נעבור על $2 \cdot \log(k)$ צמתים כש k מספר הצמתים בעץ הנוכחי)

$$\sum_{k=1}^n 2 \cdot \log(k) \leq \sum_{k=1}^n 2 \cdot \log(n) = O(n \log(n))$$

3. כמתואר בגרף מטה התקבלה נוסחא ריבועית עבור מיון של מערך ממיון בסדר הפוך כמצופה מסעיף 2
 חישבנו את מספר החילופים והחיפושים ומתקבל $O(n^2)$.

